

EEE 416 (January 2025)

Microprocessor & Embedded System Laboratory

Final Project Report

Section: C1 Group: 02

IoT-Based Smart Bangla Notice Board with Real-time Calendar & Clock

Course Instructors:

Sadman Sakib Ahbab, Lecturer
Khairul Islam, Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Bashir Shahriyer Rafi

Student ID: 2006136

Signature: _____

Full Name: Md. Rifat Rahman

Student ID: 2006137

Signature: _____

Full Name: Samia Hossain

Student ID: 2006138

Signature: _____

Full Name: Sami Hasib Khan

Student ID: 2006139

Contents

1 Abstract	2
2 Introduction	2
3 Design.....	2
3.1 Problem Formulation (PO(b))	2
3.1.1 Identification of Scope.....	2
3.1.2 Literature Review	3
3.1.3 Formulation of Problem.....	4
3.2 Design Method (PO(a))	5
3.3 Circuit Diagram	6
3.4 Hardware Design:.....	7
3.5 Software Setup.....	8
5 Design Analysis and Evaluation	10
5.1 Novelty	10
5.2 Design Considerations (PO(c))	10
5.2.1 Considerations to public health and safety	10
5.2.2 Considerations to environment.....	10
5.2.3 Considerations to cultural and societal needs.....	10
5.3 Limitations of Tools (PO(e))	11
5.4 Impact Assessment (PO(f))	11
5.4.1 Assessment of Societal and Cultural Issues.....	11
5.4.2 Assessment of Health and Safety Issues	11
5.4.3 Assessment of Legal Issues	11
5.5 Sustainability Evaluation (PO(g))	11
6 Reflection on Individual and Team work (PO(i))	12
6.1 Individual Contribution of Each Member	12
6.2 Mode of TeamWork	12
6.3 Diversity Statement of Team	12
6.4 Log Book of Project Implementation & Individual Contribution	13
7 Communication to External Stakeholders (PO(j))	14
7.1 Executive Summary.....	14
7.2 User Manual.....	14
8 Project Management and Cost Analysis (PO(k))	14
8.1 Bill of Materials	14
9 Future Work (PO(l))	15
10 References	15
Appendix A: Full Node MCU code.....	16
Appendix B: Website HTML code.....	26

1 Abstract

The **IoT-Based Bangla Smart Notice Board with Real-Time Calendar and Clock** is a modern digital display system designed to enhance communication in educational institutions and public spaces. Powered by the ESP8266 microcontroller and LED dot matrix displays, this system simultaneously presents Bangla scrolling notices, current time, and calendar date on three separate screens. Time and date information are synchronized in real-time using Network Time Protocol (NTP) over Wi-Fi, ensuring accuracy without manual intervention. The project allows dynamic notice updates through a web interface, enabling authorized users to change messages remotely. Predefined Bangla notices relevant to institutional settings can also be displayed automatically. With its visually appealing Bangla typography, synchronized multi-panel output, and internet-based control, this project offers a cost-effective and culturally relevant solution for public announcements and information display.

2 Introduction

While English-based LED dot matrix display systems for showing time, date, and public notices are widely available, Bangla implementations remain relatively uncommon. This project addresses that gap by introducing a fully functional **Bangla Smart Notice Board with Real-Time Calendar and Clock**, developed using an ESP8266 microcontroller and Bangla-compatible LED dot matrix displays. The core functionality revolves around driving three separate displays simultaneously—one for time, one for date, and one for scrolling notices—all presented in the Bangla language.

The ESP8266, equipped with built-in Wi-Fi, connects to the internet to retrieve accurate time and date information via Network Time Protocol (NTP). This enables real-time synchronization without manual setting, making the device an effective IoT solution. In addition to displaying time and date, the system serves as a dynamic notice board, capable of showcasing preloaded or web-updated Bangla messages suitable for use in educational institutions, offices, or public areas.

The physical design of the system is compact and wall-mountable, ensuring ease of installation and usability. A custom enclosure was also developed to house all necessary components, including the ESP8266 board, power supply unit, and three 8x32 LED matrix panels. This project not only showcases technical feasibility but also emphasizes cultural relevance by promoting digital communication in the Bangla language.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

1. Hardware Components Selection:

- Identify suitable hardware components such as display screens, microcontrollers, and sensors that can accommodate Bangla text rendering and time/date display functionalities.
- Ensure compatibility and feasibility of integration within the notice board system.

2. **Bangla Text Rendering:**
 - Focus on developing software algorithms capable of accurately rendering Bangla text on digital displays.
 - Ensure legibility and readability of Bangla characters by bitmapping and updating that into Arduino library
3. **Clock System & Calendar Integration:**
 - Develop a clock system synchronized with standard timekeeping protocols to display current time accurately.
 - Ensure compatibility with the chosen hardware components and software architecture.
 - Implement a calendar feature to display the current date and month in Bangla.
4. **User Interface Design:**
 - Design a user-friendly interface that allows easy navigation and interaction with the notice board, clock, and calendar functionalities.
 - Consider usability principles to ensure accessibility for users with varying levels of technical proficiency.
 - Control the notice from website
5. **Customization Options:**
 - Provide users with the ability to input and display custom notices and messages on the notice board.
 - Include features for users to customize the appearance and layout of displayed information according to their preferences.
6. **Testing and Quality Assurance:**
 - Conduct rigorous testing to ensure the functionality, reliability, and accuracy of the notice board system.
 - Perform usability testing to gather feedback on user experience and identify areas for improvement.
 - Continuation of update time & date if wifi goes off suddenly.
7. **Scalability and Future Expansion:**
 - Consider the potential for future expansion and scalability of the notice board system.
 - Design the architecture to accommodate additional features or upgrades based on user feedback and evolving requirements.
 - Bitmapping of all the Bangla letter and sign
8. **Project Constraints and Limitations:**
 - Recognize and address constraints such as budget limitations, technical challenges, and time constraints.

3.1.2 Literature Review

Digital notice boards are increasingly used for information dissemination, but there's a gap in addressing the needs of Bangla-speaking communities. Research in Bangla language processing has advanced, focusing on efficient text rendering. However, there's limited literature on integrating Bangla language with time and date features in digital signage. Digital signage systems are effective for conveying information, but there's a lack of research

on culturally sensitive design for localized solutions. Time and date integration in digital signage is common, improving user engagement.

This project aims to fill the gap by developing a Bangla notice board system integrating time and date features. It leverages insights from Bangla language processing and digital signage systems to create an inclusive communication solution for Bangla-speaking audiences.

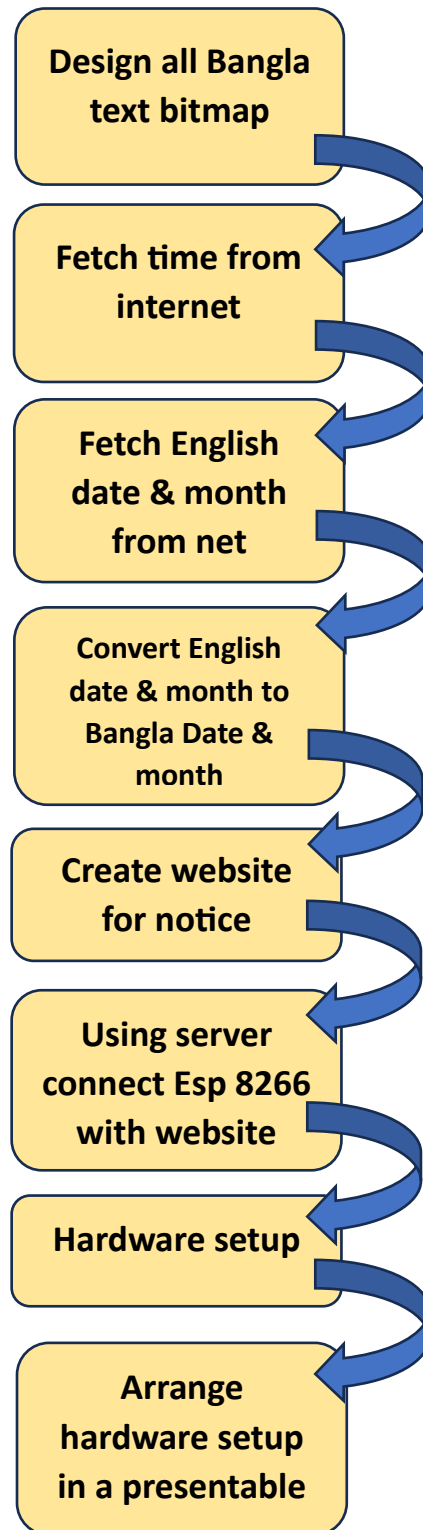
3.1.3 Formulation of Problem

We aimed to achieve some specific goals by developing the project. The objectives include:

- Developing a digital Bangla Notice Board capable of displaying text in the Bangla language.
- Implementing a calendar functionality to display current date and month in Bangla.
- Integrating a clock system to display current time in digital format alongside the notice board and calendar.
- To ensure user-friendly interaction and navigation for easy access to notice board updates, calendar, and clock functionalities.

3.2 Design Method (PO(a))

The flowchart of our work is given below:



3.3 Circuit Diagram

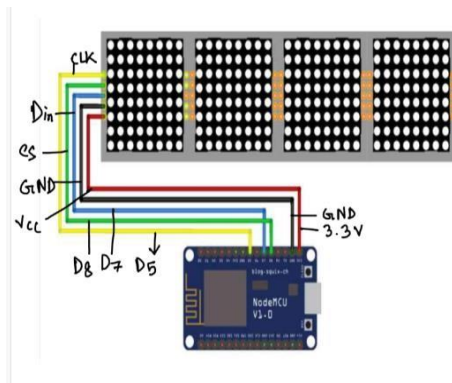


Figure 1 Connection of LED dot Matrix

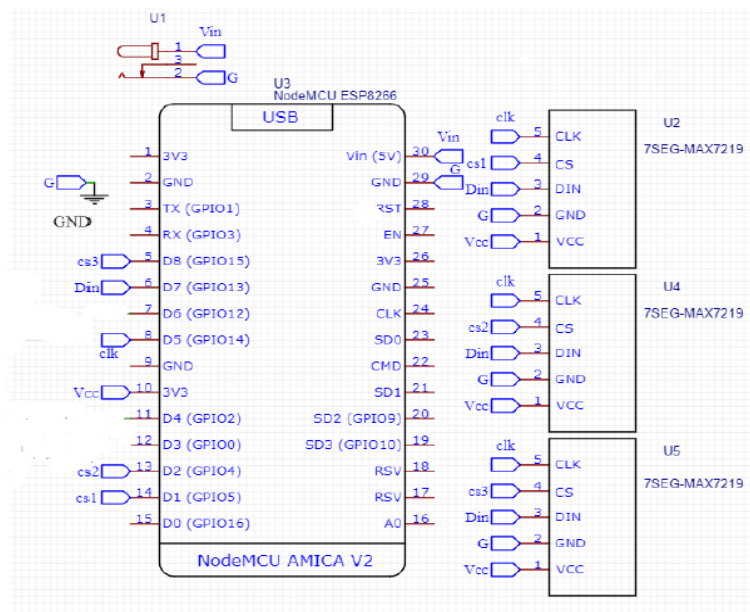


Figure 2 Connection of Node MCU

Pin diagram:

Pin	Dot matrix 01 (time)	Dot matrix 02(date & month)	Dot matrix 03 (notice)
VCC	vcc	vcc	vcc
Gnd	gnd	gnd	gnd
Din	D7	D7	D7
CS pin	D3	D8	D6
Clock	D5	D5	D5

3.4 Hardware Design:

Components:

- a. Max7219 Led Dot matrix (3x)
- b. Jumper wire
- c. Breadboard
- d. Adapter

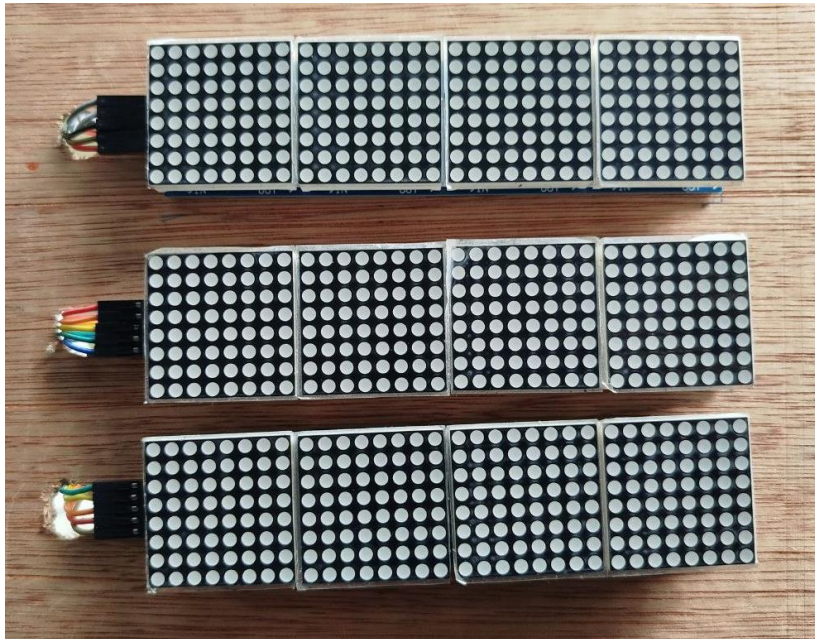


Figure 3 LED dot Matrix Display

Top Display: Time

- Shows **Hour:Minute** in real- time (e.g., ১৪:৩৫)

Middle Display: Date, Month & Week

- Displays **Bangla Date**
- **Month name** in Bangla (e.g., আষাঢ়, শ্রাবণ)
- **Day of the Week** in Bangla (e.g., বুধ, শনি)

Bottom Display: Notice Board

- Shows **scrolling messages** such as:
 - আগে নামতে দিন
 - গাড়ি থামবে
- Triggered via **web interface**

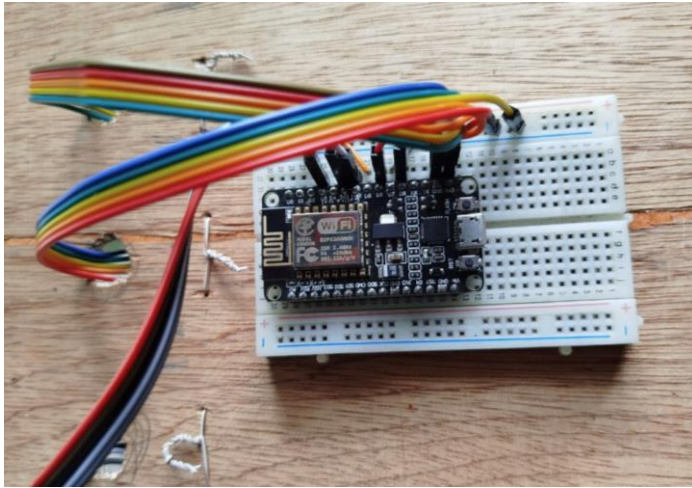


Figure 4 Main Circuit

3.5 Software Setup

we used Arduino ide for code into nodemcu.

Source code analysis:

Our code is a comprehensive ESP8266-based Bangla smart display system that:

- **Connects to Wi-Fi**
- **Displays real-time Bangla time**
- **Converts and displays Bangla date and weekday**
- **Scrolls safety/notice messages triggered via web**
- **Handles scrolling graphics using custom bitmaps**

Here's a complete breakdown of how each part works:

1. Library Includes

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include <WiFiManager.h>
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>
#include <TimeLib.h>
#include <Adafruit_GFX.h>
#include <Max72xxPanel.h>
```

These libraries enable:

- Wi-Fi communication
 - Time sync from NTP server
 - Graphics rendering on MAX7219 8x8 LED matrices
-

2. Wi-Fi Credentials & Display Pins

```
const char* ssid = "Bliss";  
const char* password = "abcdefgh";
```

Sets up Wi-Fi login.

```
#define CS_PIN_TIME D3  
#define CS_PIN_DATE D8  
#define CS_PIN D6  
#define CLK_PIN D5  
#define DATA_PIN D7
```

Defines which GPIO pins control which matrices:

- **D3 (Time), D8 (Date), D6 (Notice)**

3. Bitmaps for Bangla Months, Weekdays, Digits

```
byte boishakh1_1[8] = {...}, ... etc.
```

Defines 8x8 pixel graphics (custom Bangla letters):

- Each month is 3 tiles
- Weekdays are 2 tiles
- Digits (০ to ৯) are 1 tile

4. Matrix Initialization

```
Max72xxPanel DateMatrix(CS_PIN_DATE, 4, 1);  
Max72xxPanel TimeMatrix(CS_PIN_TIME, 4, 1);  
Max72xxPanel matrix(CS_PIN, 4, 1);
```

Each display has **4 chained 8x8 matrices** (32x8 resolution).

- matrix is for scrolling messages.

5. Web Server Setup

```
ESP8266WebServer server(80);
```

Creates a simple local web server that listens on port 80.

Handlers:

```
server.on("/", handleRoot);  
server.on("/index", handleIndex);
```

6. Bangla Message Bitmaps

```
const uint64_t IMAGES_1[] = {...};
```

Each 64-bit value represents 8x8 graphics for one character or part of a sentence.

7. Scroll Control

```
const uint64_t* currentMessage = nullptr;
int currentMessageLen = 0;
volatile bool shouldScroll = false;
```

Controls which message is currently scrolling and whether to scroll.

8. Web Interface Logic

```
void handleIndex() { ... }
```

Handles GET requests with a value param:

- "135" clears screen
- "136" to "140" select one of 5 predefined scrolling messages
- Triggers shouldScroll = true

9. Scrolling Function

```
void scrollOneFrame(...)
```

Moves message from **right to left** across the 32x8 notice display:

- Converts each uint64_t to 8-column bytes
- Slides them across the matrix

10. Time Sync via NTP

```
const char* ntpServerName = "pool.ntp.org";
configTime(0, 0, ntpServerName);
```

Fetches **UTC time from internet** and converts to **Bangladesh time** (+6 hr).

11. Bangla Date Display

```
void showBanglaDate()
```

- Converts Gregorian date to Bangla calendar
- Displays day and month using 8x8 tiles
- Then displays weekday using pre-defined tiles
- Uses slideBitmapRightToLeft() for smooth sliding

12. Bangla Time Display

```
void showTime()
```

- Fetches current hour and minute
- Converts to Bangla digit bitmaps
- Draws 4 digits (HH:MM) + colon on TimeMatrix

13. Loop Function

```
void loop() {  
  server.handleClient();  
  ...  
  showTime();  
  showBanglaDate();  
}
```

- Keeps server responsive
 - Scrolls message (if any)
 - Continuously shows time and date
-

5 Design Analysis and Evaluation

5.1 Novelty

This innovative project effectively meets key specifications while offering practical real-world applications, featuring enhanced fonts and visuals, a compact and efficient design, and cost-effective hardware. Key Novel Features are:

1. Bangla Visual Support:
Custom bitmap Bangla fonts rarely seen in such low-cost embedded systems.
2. Improved Font & Aesthetic Display:
Clearer and more readable matrix display with enhanced pixel-mapped Bangla characters.
3. Compact & Modular Design:
Three display zones (Time, Date, Notice) using daisy-chained 8x32 LED matrices.
4. Low-Cost Implementation:
Utilizes ESP8266, MAX7219, and Wi-FiManager, keeping the system both cost-effective and scalable.
5. Smart Web-Controlled Notices:
Real-time message update via web interface

5.2 Design Considerations (PO(c))

5.2.1 Considerations to public health and safety

Electrical Safety: We ensured all electrical components meet safety standards to prevent hazards.

Structural Stability: Securely mount hardware to prevent accidents.

Accessibility: Design for easy access by users of all abilities.

5.2.2 Considerations to environment

As long as we handle the power source and equipment properly by recycling and managing them well, this project won't harm the environment.

5.2.3 Considerations to cultural and societal needs

In designing the Bangla Notice Board with Time and Date Integration, it's essential to respect the cultural and societal needs of Bangla-speaking communities. This includes accurately representing the Bangla language and script, incorporating culturally relevant content and events, ensuring accessibility for all users, and respecting social norms and etiquette. Collaborating with local stakeholders, engaging community participation, and providing educational opportunities can further empower and unite the community through the notice board. By prioritizing inclusivity, diversity, and cultural sensitivity, the project can effectively serve as a platform for communication, celebration, and empowerment within Bangla-speaking communities.

5.3 Limitations of Tools (PO(e))

- Front bitmapping and padding.
- Single board implementation.
- UI design
- PCB design
- Custom kit design
- Bangla Unicode
- date & notice synchronisation

5.4 Impact Assessment (PO(f))

5.4.1 Assessment of Societal and Cultural Issues

The project makes it easier for Bangla-speaking people to access information by showing notices, dates, and times in Bangla on a digital notice board. This helps non-English speakers stay informed, encourages pride in their language, and brings communities together. It's great for places like schools or offices, where people can update messages using a website. However, the project needs clear text displays and affordable access to work well, especially in areas with limited resources

5.4.2 Assessment of Health and Safety Issues

The project is safe to use if the power source and parts are handled properly, like recycling old components. It uses safe electrical parts to avoid shocks or fires, and its small, sturdy design stays secure when placed on tables or walls. The power source keeps risks low, perfect for schools or offices. Flickering screens might be annoying but aren't dangerous. Recycling parts also protects the environment. For example, in a community center, good power management stops overheating, and proper disposal keeps things eco-friendly

5.4.3 Assessment of Legal Issues

The project follows legal rules by using approved fonts and software, protecting user data on its website, making the system accessible to everyone, and getting permits to install the notice board in public places. This keeps the project safe from legal trouble and builds trust. However, the team's limited experience might make it hard to fully understand complex rules, like font licenses or data privacy, especially with third-party tools.

5.5 Sustainability Evaluation (PO(g))

This project is sustainable by using low-power parts and encouraging recycling, but its power-hungry LED screens and unclear recycling plans could harm the environment. It helps Bangla-speaking communities by showing notices in their language, boosting cultural pride and engagement, though internet issues and a tricky interface might limit its use. It's cheap to build, great for schools or offices, but ongoing costs like battery replacements could add up. The project's creative Bangla text display and internet features allow for future upgrades, but the

small computer board struggles with bigger tasks. Getting community feedback keeps it useful, but limited testing and internet needs might make this harder.

5.6 Ethical Issue(PO(g))

Open-source libraries such as MD_Parola, Max72xxPanel, and WiFiManager were used in compliance with their licenses. No copyrighted content or unauthorized fonts were used in the system. All bitmapping for Bangla letters and digits was done manually by the team. Accuracy and honesty were maintained during documentation and testing. Cultural respect was ensured by accurately representing Bangla text and societal values. Power usage and component disposal were handled responsibly to minimize environmental harm. Equal effort and fair contribution were practiced throughout the project.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

We worked together as a team. Each member contributed to bitmapping, code writing, error debugging and hardware assembly. Every time we faced any challenge we overcame that collaboratively.

6.2 Mode of TeamWork

We did several online zoom meetings & offline meetings to come up with the design procedure and share resources to explore already existing works for smart notice boards and calendar clocks. Then we worked day and night in the hall to assemble the whole project and finally successfully implemented it.

6.3 Diversity Statement of Team

Our team brought together a diverse range of skills essential for developing the project with Real-Time Calendar and Clock. Some members focused on ESP8266 programming and Wi-Fi integration, while others worked on custom bitmapping of Bangla characters and LED matrix display control. Each member's unique strengths contributed to solving key challenges like real-time synchronization, scrolling display, and power issues. This diversity and mutual support were critical in successfully completing a culturally relevant and technically complex project.

6.4 Log Book of Project Implementation & Individual Contribution

Date	Milestone achieved	Individual Role	Team Role	Comments
20.05.2025	money collection & component buying	2006136,2006137, 2006138,2006139		
29.05.2025	Code written and uploaded for Notice Board	2006136	Observed the outcome and did debugging together later	-----
19.06.2025	Bitmapping for Bangla Notices	2006139	-----	-----
26.06.2025	Scrolling Notice & date/month observed by modifying the code	2006136 & 2006138	Done collaboratively	Faced issues with power supply, led display keeps being red
03.07.2025	Bitmapping done for Bangla calendar clock	2006139 & 2006137	-----	-----
10.07.2025	Code done for calendar clock	2006138	Done together	
17.07.2025	Code for successfully showing time	2006138 & 2006139	-----	
18.07.2025	Merging Notice Board with calendar clock	2006136		Led display and power supply problem solved
23.07.2025	Final hardware assembly done	2006136 & 2006137	Done collaboratively	

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

The Bangla Notice Board with Time and Date Integration project aims to develop a digital notice board system tailored for Bangla-speaking communities, providing essential time and date information in Bangla script. By leveraging technology to address cultural communication needs, the project promotes inclusivity and accessibility while fostering cultural pride and language preservation. Key features include accurate Bangla text rendering, synchronized clock and calendar functionalities, and user-friendly customization options. Through compliance with legal regulations, sustainability considerations, and a focus on community engagement, the project seeks to minimize risks and maximize positive impact. By enhancing communication and information dissemination within Bangla-speaking communities, the project contributes to empowerment, unity, and cultural sustainability.

7.2 User Manual

- a) Turn on the board by connecting the adapter to the power supply.
- b) Wait for the screen to stabilize and show the time & date. The time and date will take time to update.
- c) Open the interfacing website.
- d) In the website, there are buttons for some common notices. Choose from the options to show the desired notice.
- e) The top two screens will keep showing the date and time in Bangla.
- f) if you want to upload new code open the box behind the clock and using a type B port you can upload your code.

8 Project Management and Cost Analysis (PO(k))

8.1 Bill of Materials

Component Lists	Price
ESP NODE MCU	420 Tk
LED dot matrix	2005 Tk
Design	180 Tk
Power adapter	150 Tk
Total	2755 Tk

9 Future Work (PO(1))

In this project we weren't able to add the feature of typing any Bangla Notice that will be shown in LED display. Instead we made buttons for some common messages from where notices can be selected. So adding this feature is one of the most important future works for this project. To do so, Bitmapping each and every character in Bangla is needed which is a very gruesome task and it was not possible to do that within the time available in single semester. Also the date and time cannot be updated manually, it updates automatically from the server through wifi. Moreover, the board can be programmed to implement many other interesting display effects while showing the date and time

10 References

<https://dotmatrixtool.com/>

NTP Servers in Bangladesh, bd.pool.ntp.org (ntppool.org)

www.chatgpt.com

Appendix A: Full Node MCU code

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include <WiFiManager.h>
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>
#include <TimeLib.h>
#include <Adafruit_GFX.h>
#include <Max72xxPanel.h>

const char* ssid = "Bliss";
const char* password = "abcdefgh";

#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 4

#define CS_PIN_TIME D3 //4 // D3 - for time display
#define CS_PIN_DATE D8 // D8 - for date/weekday
#define CS_PIN D6 // for notice
#define CLK_PIN D5 // D5
#define DATA_PIN D7 // D7

// Web server
ESP8266WebServer server(80);

byte boishakh1_1[8] = {0x60,0x10,0xFF,0x27,0x49,0x4D,0x23,0x00};
byte boishakh1_2[8] = {0x00,0x05,0xDD,0x27,0x55,0x05,0x05,0x00};
byte boishakh1_3[8] = {0x00,0x02,0xEB,0x7A,0x0A,0x32,0x0E,0x00};
byte joishtho1_1[8] = {0x30,0x08,0x7F,0x11,0x25,0x24,0x13,0x00};
byte joishtho1_2[8] = {0x00,0x00,0xFF,0xC5,0x28,0xA4,0xA8,0x00};
byte joishtho1_3[8] = {0x1C,0x20,0xFE,0x38,0xE4,0xA4,0x78,0x00};
byte ashar1_1[8] = {0x00,0x00,0xFE,0x1B,0x56,0x46,0x3A,0x00};
byte ashar1_2[8] = {0x00,0x81,0xFD,0xA7,0x95,0xAD,0x9D,0x00};
byte ashar1_3[8] = {0x00,0x00,0xFE,0x40,0x4C,0x44,0x38,0x10};
byte srabon1_1[8] = {0x00,0x01,0x37,0x09,0x15,0x01,0x23,0x1D};
byte srabon1_2[8] = {0x00,0x40,0x7F,0xCE,0x52,0x5A,0x46,0x00};
byte srabon1_3[8] = {0x00,0x08,0x6C,0x98,0xC8,0x08,0x08,0x00};
byte bhadro1_1[8] = {0x00,0x00,0x0F,0x00,0x04,0x02,0x01,0x00};
byte bhadro1_2[8] = {0x00,0x08,0xEF,0x59,0x89,0x69,0xCA,0x01};
byte bhadro1_3[8] = {0x00,0x00,0xF0,0x00,0x20,0xD0,0x90,0x60};
byte ashin1_1[8] = {0x00,0x00,0xFE,0x1B,0x56,0x46,0x3A,0x00};
byte ashin1_2[8] = {0x1E,0xA1,0xFB,0xA4,0xAA,0xA1,0xA2,0x01};
byte ashin1_3[8] = {0x00,0x80,0xFF,0x82,0xBA,0xA6,0xB2,0x80};
byte kartik1_1[8] = {0x00,0x00,0xFE,0x3D,0x4A,0x6A,0x18,0x00};
byte kartik1_2[8] = {0x1C,0xA3,0xFF,0xA3,0xAA,0xA8,0xA7,0x00};
byte kartik1_3[8] = {0x80,0x00,0xFF,0x1E,0xA5,0xB5,0x0C,0x00};
```

```

byte ogrohayon1_1[8] = {0x00,0x00,0x1F,0x03,0x0A,0x08,0x07,0x00};
byte ogrohayon1_2[8] = {0x00,0x00,0xF6,0x49,0xC4,0xCC,0x51,0x0E};
byte ogrohayon1_3[8] = {0x00,0x80,0xD8,0x98,0x80,0x98,0x98,0x00};
byte poush1_1[8] = {0x00,0x00,0x0E,0x05,0x08,0x08,0x04,0x00};
byte poush1_2[8] = {0xB8,0x54,0xDF,0x35,0xD4,0x95,0x14,0x00};
byte poush1_3[8] = {0x00,0x00,0xF0,0x20,0xA0,0x60,0xE0,0x00};
byte magh1_1[8] = {0x00,0x00,0x07,0x02,0x01,0x03,0x03,0x00};
byte magh1_2[8] = {0x00,0x10,0xDF,0x74,0x55,0xD2,0x53,0x00};
byte magh1_3[8] = {0x00,0x00,0xE0,0x40,0x40,0x40,0xC0,0x00};
byte falgun1_1[8] = {0x00,0x00,0xFE,0x4D,0x2A,0x4A,0x38,0x00};
byte falgun1_2[8] = {0x00,0x80,0xF8,0x86,0xA9,0xD4,0x8D,0x01};
byte falgun1_3[8] = {0x00,0x80,0xFF,0x81,0x9D,0x93,0x59,0xA0};
byte choitrol_1[8] = {0x06,0x01,0x0F,0x02,0x04,0x04,0x02,0x00};
byte choitrol_2[8] = {0x00,0x00,0xFF,0x81,0xF1,0x94,0xE3,0x00};
byte choitrol_3[8] = {0x00,0x00,0xF0,0xC0,0x20,0x60,0xA0,0x00};
byte shonibar[2][8] = {{0x00,0x05,0xDF,0x25,0x55,0x05,0x05,0x00},
                        {0xF0,0x08,0xFE,0x08,0x68,0x58,0x08,0x00}};

byte robibar[2][8] = {{0x01,0x02,0xFF,0x3A,0x4A,0x2A,0x5A,0x00},
                      {0xE0,0x10,0xFC,0x70,0x90,0x90,0x70,0x00}};

byte shombar[2][8] = {{0x00,0x00,0xFF,0x29,0x45,0x47,0x2D,0x00},
                      {0x00,0x40,0x7F,0xD2,0x4A,0x5E,0x5A,0x00}};

byte mongol[2][8] = {{0x04,0x03,0xFA,0x99,0x56,0xF1,0xD3,0x00},
                     {0x00,0x40,0x7F,0xC1,0x4B,0x55,0x51,0x00}};

byte budhbar[2][8] = {{0x01,0x02,0xFE,0x39,0x4A,0x39,0x14,0x0A},
                      {0x00,0x00,0xF0,0x40,0x40,0xC0,0x40,0x00}};

byte briho[2][8] = {{0x00,0x00,0xFF,0x3B,0x48,0x3B,0x11,0x1C},
                    {0x00,0x00,0xD8,0x18,0x80,0x18,0xD8,0x00}};

byte shukro[2][8] = {{0x00,0x02,0xDB,0x26,0x52,0x06,0x0A,0x05},
                     {0x00,0x00,0xFF,0x3C,0x4A,0x3A,0x98,0x68}};

byte shunno_small_1[8] = {0x18, 0x24, 0x42, 0x42, 0x42, 0x42, 0x24, 0x18};
byte ek_small_1[8] = {0x60, 0x70, 0x1c, 0x06, 0x02, 0x62, 0x76, 0x3c};
byte dui_small_1[8] = {0x60, 0x7c, 0x06, 0x06, 0x7c, 0x60, 0x30, 0x1e};
byte tin_small_1[8] = {0x0c, 0x5a, 0x5a, 0x42, 0x42, 0x42, 0x42, 0x3c};
byte char_small_1[8] = {0x3c, 0x66, 0x66, 0x3c, 0x3c, 0x66, 0x66, 0x3c};
byte pach_small_1[8] = {0x30, 0x38, 0x46, 0x48, 0x4a, 0x46, 0x62, 0x3c};
byte chhoy_small_1[8] = {0x18, 0x58, 0x48, 0x4a, 0x4e, 0x42, 0x62, 0x3e};
byte shaat_small_1[8] = {0x38, 0x44, 0x44, 0x3c, 0x04, 0x04, 0x06, 0x06};
byte aat_small_1[8] = {0x60, 0x60, 0x26, 0x22, 0x3e, 0x24, 0x24, 0x38};
byte noy_small_1[8] = {0x60, 0x78, 0x0c, 0x06, 0x72, 0x4a, 0x6a, 0x0e};

```

```

//int pinCS = 4; // D2 for Max72xxPanel
int numberOfHorizontalDisplays = 4;
int numberOfVerticalDisplays = 1;

Max72xxPanel DateMatrix(CS_PIN_DATE, numberOfHorizontalDisplays,
numberOfVerticalDisplays); // D8
Max72xxPanel TimeMatrix(CS_PIN_TIME, numberOfHorizontalDisplays,
numberOfVerticalDisplays); // D3
Max72xxPanel matrix = Max72xxPanel(CS_PIN,numberOfHorizontalDisplays,
numberOfVerticalDisplays);//D6

// Sentence 1: line mene cholun
const uint64_t IMAGES_1[] = {
    0x3f203f2428242926, 0x20212d73b3aba020, 0x203f242828262020,
    0x0000000000000000, 0x3f22242e36312926, 0x3f24282826312926,
    0x0000000000000000, 0x2020242a293f2020, 0x21223f2b3328322c,
    0x203f242828262020
};
const int LEN_1 = sizeof(IMAGES_1) / 8;

// Sentence 2: aage namte din
const uint64_t IMAGES_2[] = {
    0x3f3f222c3229212e, 0x3f10262a1a21322c, 0x0000000000000000,
    0x3f203f2428282620, 0x203f24242e362020, 0x2e31352922342926,
    0x0000000000000000, 0x2020afa8a4bea07f, 0x20203f2428282620
};
const int LEN_2 = sizeof(IMAGES_2) / 8;

// Sentence 3: jatra shuvo hok
const uint64_t IMAGES_3[] = {
    0x7f407f424a566240, 0x7f405f626252424c, 0x0000000000000000,
    0x4142ff453b384030, 0x5c7249697979425c, 0x0000000000000000,
    0x7f5865635360514e, 0x4c52507f514a4642
};
const int LEN_3 = sizeof(IMAGES_3) / 8;

// Sentence 4: nirapod durotto bojay rakhun
const uint64_t IMAGES_4[] = {
    0x20bfa4a8a8a6603f, 0x3f203f322b262220, 0x20203f1828241808,
    0x20202f28243c2020, 0x0000000000000000, 0x21222f2b243e2020,
    0x20203f322b262220, 0x202b353129222c20, 0x0000000000000000,
    0x20203f322a262220, 0x3f2728373539222c, 0x20203f222b362220,
    0x0000000000000000, 0x3f203f322b262220, 0x2e223f011d232d14,
    0x203f242828262020
};
const int LEN_4 = sizeof(IMAGES_4) / 8;

// Sentence 5: gari thambe

```

```

const uint64_t IMAGES_5[] = {
    0x3f203f10202c3414, 0x2caaaabba2bc603f, 0x0000000000000000,
    0x3f203f02122a261a, 0x203f24242e362020, 0x3f29262231292620
};
const int LEN_5 = sizeof(IMAGES_5) / 8;

// Global scroll state
const uint64_t* currentMessage = nullptr;
int currentMessageLen = 0;
int scrollSpeed = 80;
volatile bool shouldScroll = false;

// Web root
void handleRoot() {
    server.send(200, "text/plain", "ESP8266 is running");
}

// Web message trigger
void handleIndex() {
    if (server.hasArg("value")) {
        String value = server.arg("value");
        Serial.print("Received Index: ");
        Serial.println(value);

        shouldScroll = false; // Stop previous scroll

        if (value == "135") {
            matrix.fillScreen(LOW);
            matrix.write();
            currentMessage = nullptr;
            currentMessageLen = 0;
            server.send(200, "text/plain", "Display Cleared");
            return;
        } else if (value == "136") {
            currentMessage = IMAGES_1;
            currentMessageLen = LEN_1;
        } else if (value == "137") {
            currentMessage = IMAGES_2;
            currentMessageLen = LEN_2;
        } else if (value == "138") {
            currentMessage = IMAGES_3;
            currentMessageLen = LEN_3;
        } else if (value == "139") {
            currentMessage = IMAGES_4;
            currentMessageLen = LEN_4;
        } else if (value == "140") {
            currentMessage = IMAGES_5;
            currentMessageLen = LEN_5;
        }
    }
}

```

```

    } else {
        server.send(400, "text/plain", "Invalid value!");
        return;
    }

    shouldScroll = true;
    scrollSpeed = 80;
    server.send(200, "text/plain", "OK: " + value);
} else {
    server.send(400, "text/plain", "Missing 'value' parameter");
}
}

// Scroll one frame across full 32x8 display
void scrollOneFrame(const uint64 t *images, int len, int shift) {
    const int charWidth = 8;
    const int totalWidth = len * charWidth;

    static uint8 t buffer[256]; // up to 32 characters
    for (int i = 0; i < len; i++) {
        uint64 t img = images[i];
        for (int col = 0; col < charWidth; col++) {
            buffer[i * charWidth + col] = (img >> (col * 8)) & 0xFF;
        }
    }

    matrix.fillScreen(LOW);
    for (int x = 0; x < numberOfHorizontalDisplays * 8; x++) {
        int sourceCol = shift + x;
        if (sourceCol >= 0 && sourceCol < totalWidth) {
            for (int y = 0; y < 8; y++) {
                bool pixelOn = bitRead(buffer[sourceCol], y);
                matrix.drawPixel(numberOfHorizontalDisplays * 8 - 1 - x, y, pixelOn);
            }
        }
    }
    matrix.write();
}

//Max72xxPanel GD(pinCS, numberOfHorizontalDisplays, numberOfVerticalDisplays);
//const char* ssid = "iPhone";
//const char* password = "samia1491";
//const char* ssid = "Sami's realme 8";
//const char* password = "sami2006139";

const char* ntpServerName = "pool.ntp.org";
const long timezoneOffset = 6 * 3600; // +6 hours

// === MAPPING FUNCTIONS ===

```

```

byte* getBanglaDigit(int digit) {
    switch (digit) {
        case 0: return shunno_small_1;
        case 1: return ek_small_1;
        case 2: return dui_small_1;
        case 3: return tin_small_1;
        case 4: return char_small_1;
        case 5: return pach_small_1;
        case 6: return chhoy_small_1;
        case 7: return shaat_small_1;
        case 8: return aat_small_1;
        case 9: return noy_small_1;
        default: return shunno_small_1;
    }
}

byte** getBanglaMonth(int bMonth) {
    static byte* months[][3] = {
        {},
        {boishakh1_1, boishakh1_2, boishakh1_3},
        {joishtho1_1, joishtho1_2, joishtho1_3},
        {ashar1_1, ashar1_2, ashar1_3},
        {srabon1_1, srabon1_2, srabon1_3},
        {bhadro1_1, bhadro1_2, bhadro1_3},
        {ashin1_1, ashin1_2, ashin1_3},
        {kartik1_1, kartik1_2, kartik1_3},
        {ogrohayon1_1, ogrohayon1_2, ogrohayon1_3},
        {poush1_1, poush1_2, poush1_3},
        {magh1_1, magh1_2, magh1_3},
        {falgun1_1, falgun1_2, falgun1_3},
        {choitro1_1, choitro1_2, choitro1_3},
    };
    return months[bMonth];
}

byte** getBanglaWeekday(int weekDay) {
    static byte* weekdays[][2] = {
        {},
        {shombar[0], shombar[1]},
        {mongol[0], mongol[1]},
        {budhbar[0], budhbar[1]},
        {briho[0], briho[1]},
        {shukro[0], shukro[1]},
        {shonibar[0], shonibar[1]},
        {robibar[0], robibar[1]}
    };
    return weekdays[weekDay];
}

```

```

// === SETUP ===
void setup() {
  Serial.begin(115200);

  // Init both matrices
  DateMatrix.setIntensity(5);
  TimeMatrix.setIntensity(5);
  matrix.setIntensity(5);
  // adjust if text appears flipped or rotated
  matrix.fillScreen(LOW);
  matrix.write();

  for (int i = 0; i < numberOfHorizontalDisplays; i++) {
    // DateMatrix.setRotation(i, 1);
    TimeMatrix.setRotation(i, 1);
    DateMatrix.setRotation(i, 1); // Rotate each panel clockwise
    matrix.setRotation(i, 1);
  }

  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(5);
    Serial.print(".");
  }
  Serial.println("WiFi connected");
  Serial.println();
  Serial.print("Connected! IP address: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/index", handleIndex);
  server.begin();
  Serial.println("HTTP server started");

  configTime(0, 0, ntpServerName); // Use UTC
  while (!time(nullptr)) {
    delay(10);
    Serial.print(".");
  }
  Serial.println("Time set");
}

void slideBitmapRightToLeft(byte** bitmaps, int count, int totalWidth) {
  for (int shift = 0; shift <= totalWidth; shift++) {
    DateMatrix.fillScreen(LOW);
    int x = 32 - shift;

```

```

    for (int i = 0; i < count; i++) {
        DateMatrix.drawBitmap(x + i * 8, 0, bitmaps[i], 8, 8, HIGH);
    }
    DateMatrix.write();
    delay(60);
}
}

void showBanglaDate() {
    time_t now = time(nullptr);
    now += timezoneOffset;
    struct tm *timeinfo = gmtime(&now);

    int gDay = timeinfo->tm_mday;
    int gMonth = timeinfo->tm_mon + 1;
    int gYear = timeinfo->tm_year + 1900;
    int weekDay = timeinfo->tm_wday;
    if (weekDay == 0) weekDay = 7;

    int bDay = gDay, bMonth = gMonth, bYear = gYear;
    if (gMonth >= 4 && gMonth <= 12) bYear = gYear - 593;
    else bYear = gYear - 594;

    if (gMonth == 4) { if (gDay < 14) { bMonth = 12; bDay = gDay + 17; } else { bMonth = 1; bDay =
gDay - 13; } }
    else if (gMonth == 5) { if (gDay < 15) { bMonth = 1; bDay = gDay + 17; } else { bMonth = 2; bDay =
gDay - 14; } }
    else if (gMonth == 6) { if (gDay < 15) { bMonth = 2; bDay = gDay + 17; } else { bMonth = 3; bDay =
gDay - 14; } }
    else if (gMonth == 7) { if (gDay < 16) { bMonth = 3; bDay = gDay + 16; } else { bMonth = 4; bDay =
gDay - 15; } }
    else if (gMonth == 8) { if (gDay < 16) { bMonth = 4; bDay = gDay + 16; } else { bMonth = 5; bDay =
gDay - 15; } }
    else if (gMonth == 9) { if (gDay < 16) { bMonth = 5; bDay = gDay + 16; } else { bMonth = 6; bDay =
gDay - 15; } }
    else if (gMonth == 10) { if (gDay < 16) { bMonth = 6; bDay = gDay + 15; } else { bMonth = 7; bDay
= gDay - 15; } }
    else if (gMonth == 11) { if (gDay < 15) { bMonth = 7; bDay = gDay + 16; } else { bMonth = 8; bDay
= gDay - 14; } }
    else if (gMonth == 12) { if (gDay < 15) { bMonth = 8; bDay = gDay + 16; } else { bMonth = 9; bDay
= gDay - 14; } }
    else if (gMonth == 1) { if (gDay < 14) { bMonth = 9; bDay = gDay + 17; } else { bMonth = 10; bDay
= gDay - 13; } }
    else if (gMonth == 2) { if (gDay < 13) { bMonth = 10; bDay = gDay + 18; } else { bMonth = 11; bDay
= gDay - 12; } }
    else if (gMonth == 3) { if (gDay < 15) { bMonth = 11; bDay = gDay + 17; } else { bMonth = 12; bDay
= gDay - 14; } }

    // ===== DAY AND MONTH SLIDE =====

```

```

byte* dateBitmaps[5];
int count = 0;

if (bDay >= 10) {
    dateBitmaps[count++] = getBanglaDigit(bDay / 10);
    dateBitmaps[count++] = getBanglaDigit(bDay % 10);
} else {
    dateBitmaps[count++] = getBanglaDigit(bDay);
}

byte** monthBitmap = getBanglaMonth(bMonth);
for (int i = 0; i < 3; i++) {
    dateBitmaps[count++] = monthBitmap[i];
}

slideBitmapRightToLeft(dateBitmaps, count, count * 8);
//delay(1);

// ===== WEEKDAY SLIDE =====
byte** weekdayBitmap = getBanglaWeekday(weekDay);
byte* weekSlide[2] = { weekdayBitmap[0], weekdayBitmap[1] };
slideBitmapRightToLeft(weekSlide, 2, 2 * 8);
//delay(2000);
}

void showTime() {
    time_t now = time(nullptr);
    now += timezoneOffset; // Convert to BD time
    struct tm *timeinfo = gmtime(&now);

    int h = timeinfo->tm_hour;
    int m = timeinfo->tm_min;

    TimeMatrix.fillScreen(LOW);
    TimeMatrix.drawBitmap(0, 0, getBanglaDigit(h / 10), 8, 8, HIGH);
    TimeMatrix.drawBitmap(8, 0, getBanglaDigit(h % 10), 8, 8, HIGH);
    // Draw colon manually at center
    TimeMatrix.drawPixel(16, 2, HIGH);
    TimeMatrix.drawPixel(16, 5, HIGH);
    //
    TimeMatrix.drawBitmap(16, 0, getBanglaDigit(m / 10), 8, 8, HIGH);
    TimeMatrix.drawBitmap(24, 0, getBanglaDigit(m % 10), 8, 8, HIGH);
    TimeMatrix.write();
}

void loop() {
    server.handleClient();
}

```

```
if (shouldScroll && currentMessage != nullptr) {  
    const int charWidth = 8;  
    int totalWidth = currentMessageLen * charWidth;  
    static int shift = -numberOfHorizontalDisplays * 8;  
  
    scrollOneFrame(currentMessage, currentMessageLen, shift++);  
    if (shift > totalWidth) shift = -numberOfHorizontalDisplays * 8;  
  
    //delay(8000);  
}  
showTime();    // Show on time matrix (D3)  
//delay(50);  
showBanglaDate(); // Show on date matrix (D8)  
//delay(10);  
}
```

Appendix B: Website HTML code

```
<!DOCTYPE html>
<html lang="bn">
<head>
  <meta charset="UTF-8">
  <title>Smart Bangla Notice Board</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #fdfdfd;
      color: #222;
      text-align: center;
      padding: 20px;
    }

    h1 {
      color: #333;
      margin-bottom: 10px;
    }

    .button-group {
      margin-top: 25px;
    }

    .button-group input[type="button"] {
      margin: 5px;
      padding: 10px 20px;
      font-size: 16px;
      background-color: #e8e8e8;
      border: 1px solid #ccc;
      border-radius: 5px;
      cursor: pointer;
    }

    .button-group input[type="button"]:hover {
      background-color: #dcdcdc;
    }

    #selected-msg {
      margin-top: 30px;
      padding: 12px 20px;
      font-size: 20px;
      color: #0047ab;
      font-weight: bold;
      background-color: #eef4ff;
      border: 1px solid #b3d1ff;
      border-radius: 6px;
      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.08);
      display: inline-block;
    }

    #project-info {
      margin-top: 20px;
      font-size: 18px;
    }

    #login-box {
      margin-top: 100px;
    }

    #password {
      padding: 8px;
      font-size: 16px;
```

```

width: 200px;
}

#login-button {
padding: 8px 16px;
font-size: 16px;
margin-left: 10px;
cursor: pointer;
}
</style>

<script>
const password = "micro416";

function login() {
const input = document.getElementById("password").value;
if (input === password) {
document.getElementById("login-box").style.display = "none";
document.getElementById("main-content").style.display = "block";
} else {
alert("Wrong password!");
}
}

function sendIndex(index, label) {
const espIP = "http://192.168.171.136"; // Change this to your actual ESP IP
fetch(`${espIP}/index?value=${index}`)
.then(response => response.text())
.then(data => {
console.log("ESP Response:", data);
})
.catch(error => {
console.error("Error:", error);
});

// Update selected message display
document.getElementById("selected-msg").innerText = "Selected: " + label;
}
</script>
</head>

<body>

<!-- Login Section -->
<div id="login-box">
<h2>Enter Password</h2>
<input type="password" id="password" placeholder="Enter password">
<button id="login-button" onclick="login()">Login</button>
</div>

<!-- Main App Content -->
<div id="main-content" style="display:none;">
<h1>Bangla Smart Notice Board</h1>

<!-- Project Info -->
<div id="project-info">
<input type="text" value="Project 416 by Group2, C1" readonly>
</div>

<!-- Button Group -->
<div class="button-group">
<input type="button" value="লাইন মেনে চলুন" onclick="sendIndex(136, 'লাইন মেনে চলুন')">
<input type="button" value="আগে নামতে দিন" onclick="sendIndex(137, 'আগে নামতে দিন')">
<input type="button" value="যাত্রা শুভ হোক" onclick="sendIndex(138, 'যাত্রা শুভ হোক')">
<input type="button" value="নিরাপদ দুরন্ত বজায় রাখুন" onclick="sendIndex(139, 'নিরাপদ দুরন্ত বজায় রাখুন')">
<input type="button" value="গাড়ি থামবে" onclick="sendIndex(140, 'গাড়ি থামবে')">

```

```
<input type="button" value="Clear" onclick="sendIndex(135, 'Clear Notice')">
</div>
```

```
<!-- Selected Message Display -->
<div id="selected-msg">Selected: None</div>
</div>
```

```
</body>
</html>
```